

NLP

Next revolution will come when we are able to interact with machines the same way we interact with humans.

The functionality of search engines is powered by NLP. Other modern applications include

- 1) Contextual Advertisements
- 2) Social Media: removing adult content, opinion mining
- 3) Search Engines.
- 4) Chatbots. (let's say in Swiggy or Amazon)

: Customer Service
Try thinking more user yourself!!

Natural Language Processing is a branch of artificial Intelligence (AI) that helps machines to understand and process human languages either in text or audio form.

Approaches to NLP

1) Heuristic Approach

"Jugaad" approach

Rule based approaches were created by NLP practitioners.

Eg: let's say for sentiment analysis for a movie review, we count the number of positive words & no. of negative words.

Examples:

- 1) Regular Expressions
- 2) Word Net
- 3) Open Mind Common Sense.
 - ↳ Open ^{community} for everyone (open source)
 - ↳ Anyone can contribute common English facts. Basically a database was created to be used for NLP tasks.

These are quick approaches, error free because humans are constructing it.

2) Machine Learning Approach

Good for open ended problems. We cannot make general rules everywhere.

Here, our algorithms construct the internal patterns required b/w Input and Output.

ML Algos used, 1) Naive Bayes

2) SVM

3) Logistic Regression

4) LDA

5) Hidden Markov Models. (Or) others.

They don't care about sequential data → which is fixed by RNN and

Bottleneck of

Heuristic Approach.

3) Deep learning Approach

→ Automatic feature and parameter generation.

1) RNN

2) LSTM

3) GRU/CNN

4) Transformers.

5) Auto encoders.

NLP Pipeline

The following steps are necessary for constructing any NLP application

1) Data Acquisition: Acquiring relevant data
Even if we use Heuristic Approach, we need extra data for better performance. (More data is always good)

2) Text Preparation: Eg.

i) Text cleanup: fixing Spelling mistakes, incomplete phrases.

ii) Basic preprocessing: ^{Eg:} Tokenization of words.

iii) Advanced preprocessing

Eg: Parts of Speech Tagging, Chunking.

3) Feature Engineering: We need data in correct format
We need to change our text data into numerical data. Eg: Word2Vec, TF-IDF, One-hot encoding.

4) Modelling: Actual Algorithm is applied

5) Deployment: Monitoring & Model Update.

The above pipeline is not universal. For elementary NLP tasks like POS, Text Representation, this works fairly well.

Data Acquisition

Available with you

Data Available but not with you (Some other company)

No one has the data then Start Collecting data (You are the first one to work on this) and manually label them.

If its less in quantity, we have to go for

"DATA AUGMENTATION" let's say

1) Replacing words with its synonyms.

2) Back translate: Convert words from Eng → Hindi and then Hindi → Eng. The sentence structure changes in this process.

3) Add noise: Adding words to the sentence without changing the sentiment of the sentence

4) Bigram flip:

"I am walking on the road"
↓
"I am walking the on road"

Like public dataset github/haggle/ some major university
↳ webscrapping → using beautiful soup (Python library)
↳ API → Use Rapid API for finding APIs. related to your domain
↳ pdf/Image → Some python library
↳ Audio → Audio to Text then start pre-processing

Text Preparation

- Cleaning
 - ↳ HTML tag removal
 - ↳ emoji from social media are removed
 - ↳ Spelling checker
- None of them are mandatory (task dependent) (lib called Textblob is used)

Basic preprocessing:

- Basic
 - Tokenization (word & sentence Tokenization)

Optimal

- Stop word Removal.

(., , , ,)

don't have semantic

- Stemming

(dance, dancing, danced)

(brings words to their root form)

(All 3 of them have same semantic meaning)

- Removing digits, punctuations

- Lowercasing

- Language detection.

Advanced Preprocessing

- POS tagging

- Parsing

- Coreference resolution

If we do stop word removal, we can't do parts of speech tagging (POS). So, we have to be very careful & modify our pipeline based on our end goal.

Feature Engineering

- When we are using Machine learning approach
 - * We have to do the feature engg. ourselves.
 - (Model's Interpretability is high)
- Deep L approach
 - * Feature engg. is done by the neural networks.
 - (Model's Interpretability is low)

Modelling

- modelling
 - data is ready, now we have to fit a model to find patterns.

evaluation

- intrinsic (model centric)
- extrinsic (business why product is centric) not being used?? by

google (let's say auto complete)

possible ways:

heuristic: if data is less, we fit a general rule to define trends.

ML Algo: Data (more data)

DL Algo: Data (even more data) → use Transfer Learning.

(BERT)

We have a model trained on a very large dataset and it has large capabilities.

Cloud API:

Decided by nature of data & Nature of Problem

let's say we are making a spam email classifier

for heuristic approach, we take a note of the email id from which we are continuously receiving spam emails

Text Preprocessing

Some of the common text preprocessing/cleaning steps:

- * Lower casing
- * Removal of Punctuations
- * Removal of Stop Words
- * Removal of Frequent Words
- * Removal of Rare Words
- * Stemming
- * Lemmatization
- * Removal of emojis / emoticons
- * Conversion of emoticons to words
- * Removal of URLs
- * Removal of HTML tags
- * Chat words conversion
- * Spelling correction

Check Out

"Text-Preprocessing - AlWinter.ipynb"

Feature Engineering / Text Representation

We convert texts into numbers (numerical data)

those are represented by vectors

So, this step is also called "text vectorization"

The numbers should capture the semantic meaning of the text

(hidden meanings should be represented)



Terms:

1) Corpus (C): when we concatenate all of our strings
(let's say we have movie reviews.)

When we combine all movie reviews together (many words might be repeating)

2) Vocabulary (V): Unique words from corpus.

3) Document (D): each movie review

4) Word (w): Each word of document

Eg: Review 1: "I loved SRK in Jawaan"

Review-2: "All actors had great chemistry"

Review 3: "Boring & predictable story line"

Sentiment

1

1

0

I haven't removed stop words here
Do remove it if required

Corpus (C): I loved SRK in Jawaan All Actors had great chemistry Boring & predictable story line. → 15 words.

Vocabulary (V): 15 words (unique words)

Documents (D): Rev 1, Rev 2, Rev 3

① One hot encoding

S2: My college is "TK" Vocabulary:

S3: College is 11TK 0 11TK, is, my, college

]" stop words
not removed
You might
choose to
remove"

We represent words in terms of vectors of length V .

$$V = 4$$

$11TK$ is my college
 $11TK = \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}$
 is = $\begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix}$
 my = $\begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix}$
 college = $\begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}$

S_0, S_1 is converted to: $\begin{bmatrix} [1, 0, 0, 0], [0, 1, 0, 0], [0, 0, 1, 0], [0, 0, 0, 1] \end{bmatrix}$ 4×4

S_3 : $\begin{bmatrix} [0, 0, 0, 1], [0, 1, 0, 0], [1, 0, 0, 0] \end{bmatrix}$ 3×4

Very logical, intuitive, easy to code.

Flaws: 1) Sparsity

Let's say our vocabulary is 50000 (corpus of some million words)

[illegible]

leads to overfitting (lots of zeroes in it)

2) Sentence is represented by Words X Vocabulary has to be kept same for any ML Model.
All sentences don't have same size. Input size

3) Out of Vocabulary (OOV) Problem:

let's say we have to predict the sentiment of the sentence:

"IITB is better"

is better out of Vocabulary words So, we want predict the Sentiment.

Let's say:

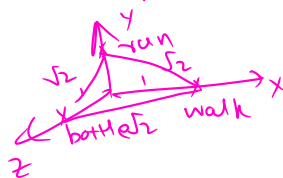
walk, run, bottle = Vocabulary

walk: $[1, 0, 0]$

$\gamma_{um} = [0, 1, 0]$

bottle: $[0, 0, 1]$

Vector Representation



So, distance b/n run-walk
= dist. b/n bottle walk.

But, run & walk have a closer meaning compared to the other pair.

So, the Semantic meaning is not being captured, which kills our purpose of vectorization.

2 Bag of Words (Bow)

Core Idea:

Sentences with same semantic meaning will have same type of words that appear in similar frequencies in the sentence.

Order of the words does not matter

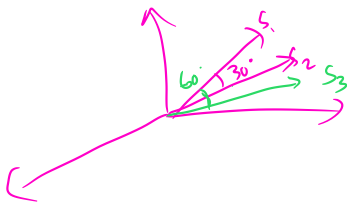
content does not matter (but at the end it's captured individually while writing the vectors)

All text is converted to V-dimensional vector.

Let's say $V=3$

Sentence is given a vector in the V-dimensional

If the dot product is high, they are similar to each other.



Let's $\angle$ b/w $S_1, S_2 = 30^\circ$

$\angle$ b/w $S_1, S_3 = 60^\circ$

$$S_1 \cdot S_2 = \cos 30^\circ = \frac{\sqrt{3}}{2}$$

$$S_1 \cdot S_3 = \cos 60^\circ = \frac{1}{2}$$

S_1, S_2 : more similar

(Because they are close to each other.)
 $\angle$ is lesser.

Scikit Learn has

{ count - vect }

functionality

max - features

binary generally
[binary = false;]

if true

then freq > 1 will be made 1

vector [1, 0, 1, 2, 3] $\rightarrow$ [1, 0, 1, 1, 1]

S_1 : Banden is a boy

S_2 : Banden is Banden

S_3 : Boys hate Banden

Boy

freq. of boy in S_1

Banden is a hate

1

1 1 0

0

2 1 0 0

1

1 0 0 1

Vocabulary = 5

So, 5^(V) dimensional vectors for each sentence

max - features: when many rare words are present. It decides

the number of words considered for analysis

max - features = 1 So, top freq. words

max - features = 2 So, top 2 freq. words

are analyzed.

Bottlenecks:

1) Sparsity: The dim. of vectors

will be very large (let's say 50,000)

So, many zeroes will be present, end up overfitting.

2) OOV: Improvement from OHE

This technique won't report an error.

3) Ordering: We are dealing with frequencies

4) S_1 : Banden is a very good singer and a nice boy

S_2 : Banden is not a very good singer and a nice girl.



Vectors might be close but semantic meaning is very different.

3 Bag of n-grams

Single word were creating issues
'not good' and 'good' have different meanings

$n=2$ (Bigram)

Example We take pairs.

S_1 : Barudev is your mentor

S_2 : Mansi is your mentor

S_3 : khushal is your mentor and mansi is your mentor.

I haven't removed stop words here, remove it if required

Tokens Vocab:

Barudev is | is your | your mentor | Mansi is | khushal is | mentor and | and Mansi

S_1 :	1	1	1	0	0	0	0
S_2 :	0	1	1	1	0	0	0
S_3 :	0	1	1	0	1	1	1

We used bigram that's why we can create ^{greater} angles b/w 'not good' & 'good'.
If we use trigrams, difference would be more clear.
So, Semantic meaning is captured better of the sentence

Benefits

- 1) Easy to implement
- 2) Able to capture

Disadvantages

- 1) We still ignore OOV
- 2) uni-gram
bi-gram
tri-gram
dimensionality ↑
as token increase
So, this increases the model training time.

4 Tf - Idf

Term frequency

Inverse-document frequency

Core Intuition:

If a word appears ^(rare) less in the corpus, then its weight increases in the sentence its present in.

S_1 : Good old Barudev boy

S_2 : Barudev nice boy

S_3 : Old Barudev good boy

$$TF(\text{Barudev}) = \frac{1}{6}, IDF(\text{Barudev}) = \ln\left(\frac{3}{3}\right) = 0$$

$$TF(\text{nice}) = 0, IDF(\text{nice}) = \ln\left(\frac{3}{1}\right) = \ln 3$$

rare word [Here, Stop words have been removed]

nice is rare in the corpus.

(Rare words will have higher IDF)

$$TF(t, d) = \frac{\text{No. of occurrences of term } t \text{ in doc } d}{\text{Total no. of terms in doc } d}$$

$0 < TF < 1$ (Probability) $\left\{ \begin{array}{l} \text{If } TF \uparrow : \text{So, term } t \text{ is} \\ \text{more frequent} \end{array} \right.$

$$IDF = \ln \left(\frac{\text{Total no. of doc. in corpus}}{\text{No. of docs with term } t \text{ in them}} \right)$$

Why $\ln$ is used?

$\text{Idf} = \left(\frac{N}{n}\right) ??$

If $N = 10000$
 $n = 2$

then $\frac{N}{n} = 5000$ value is very large

So, when we are considering $\text{IDF} * \text{TF}$,
larger IDF values will dominate over TF values.
We want both of them to contribute equally.

So, $\text{IDF} = \log_e\left(\frac{N}{n}\right) = \ln(5000)$
Here, $\log_e$ normalizes the value

Sometimes, $\text{IDF} = \ln\left(\frac{N}{n}\right) + 1$ [In sklearn library]

If any word is appearing in every sentence; its

$\text{IDF} = \ln(1) + 1 = 1$ instead of (0)

TF-IDF Calc for SI

Each term of word Vocab for corpus:
in vector is represented
by $\text{TF} * \text{IDF}$

{ Good, old, Basudev, boy, nice }

$$\text{TF}(\text{Good}) = \frac{1}{4}, \text{IDF}(\text{Good}) = \ln\left(\frac{3}{2}\right)$$

$$\text{TF}(\text{Old}) = \frac{1}{4}, \text{IDF} = \ln\left(\frac{3}{2}\right)$$

$$\text{TF}(\text{Basudev}) = \frac{1}{4}, \text{IDF} = \ln\left(\frac{3}{3}\right)$$

$$\text{TF}(\text{boy}) = \frac{1}{4}, \text{IDF} = \ln\left(\frac{3}{3}\right)$$

$$\text{TF}(\text{nice}) = 0, \text{IDF} = \ln\left(\frac{3}{1}\right)$$

So, SI is represented by

$$= \left[\frac{1}{4} \ln \frac{3}{2}, \frac{1}{4} \ln \frac{3}{2}, \frac{1}{4} \ln 1, \frac{1}{4} \ln 1, 0 \times \ln 3 \right]$$

$$= \left[\frac{\ln 1.5}{4}, \frac{\ln 1.5}{4}, 0, 0, 0 \right]$$

$$= [0.101, 0.101, 0, 0, 0]$$

Advantages

- 1) Used for Information Retrieval
Search Engines go through millions of webpages and give suggestions to you.

Disadvantages

- 1) Sparsity (High dimensions, so lots of zeroes) leads to over fitting again
- 2) OOV
- 3) Semantic Meaning is still not captured in some cases.

"Einstein was genius"

"Einstein was brilliant"

They kinda have similar meanings. But TF-IDF treats them differently.

5) Miscellaneous Custom Features

Ex $\frac{\text{No. of Positive Reviews}}{\text{No. of Negative Reviews}}$

Here, domain knowledge plays an important role.

Word Embeddings

frequency types
→ Bow
→ Tf-Idf
→ Glove

& prediction types
→ Word2Vec

Word2Vec

- 1) Semantic Meaning is captured
- 2) low dimensional vectors are formed (300-600)
- 3) dense vectors (non-zero elements)

We can use pretrained as well as self-trained word2vec models.
↓
training it on our own dataset.

↳, overfitting ↓
→ Representing the

Vectors in 300 dimensional space

→ Vector Arithmetic works here (Addition, Subtraction, Dot Product)
↳ Dot product also works.

The features would be anonymous to us.

	king	queen	Man	Woman	Monkey
gender f_1	1	0	1	0	1
Wealth f_2	1	1	0.3	0.3	0
Power f_3	1	0.7	0.2	0.2	0
weight f_4	0.8	0.4	0.6	0.5	0.3
speak f_5	1	1	1	1	0

We can never find out what they mean exactly

king = $[1, 1, 1, 0.8, 1]$

Internally, Word2Vec creates features based on the vocabulary.
These features might be different for different vocabulary.

king - man + woman $\approx$ queen

We use automated techniques to create features.

We might have a vocabulary of 50k words. So, manually generating features is quite tough

So, we use Neural Networks

All 300 numbers represent unique features, but we can never find out what they represent exactly.

Core Intuition

The underlying assumption of Word2Vec is that two words sharing similar contexts also share similar meaning and consequently a similar vector representation from the model.

Types of Word2Vec:

↳ CBow ↳ skip-gram

Continuous Bag of Words (CBow)

let's say: "Choose Architecting Intelligence for deep learning"
we have.

We want to write the word embeddings for each word.
(vectors)

let's first decide the size of these vectors for each word
(dimension)

(let it be 3)

So, choose = $[-, -, -]$

Now, We create a dummy problem and try to solve it.
 "Choose Architecting Intelligence for deep learning" → Sentence
 We start by considering windows of size=3 [It can be 5, 7, 9... also]

1st window
 central word: target word. (Architecting)
 left & right words: context words (Choose, Intelligence)

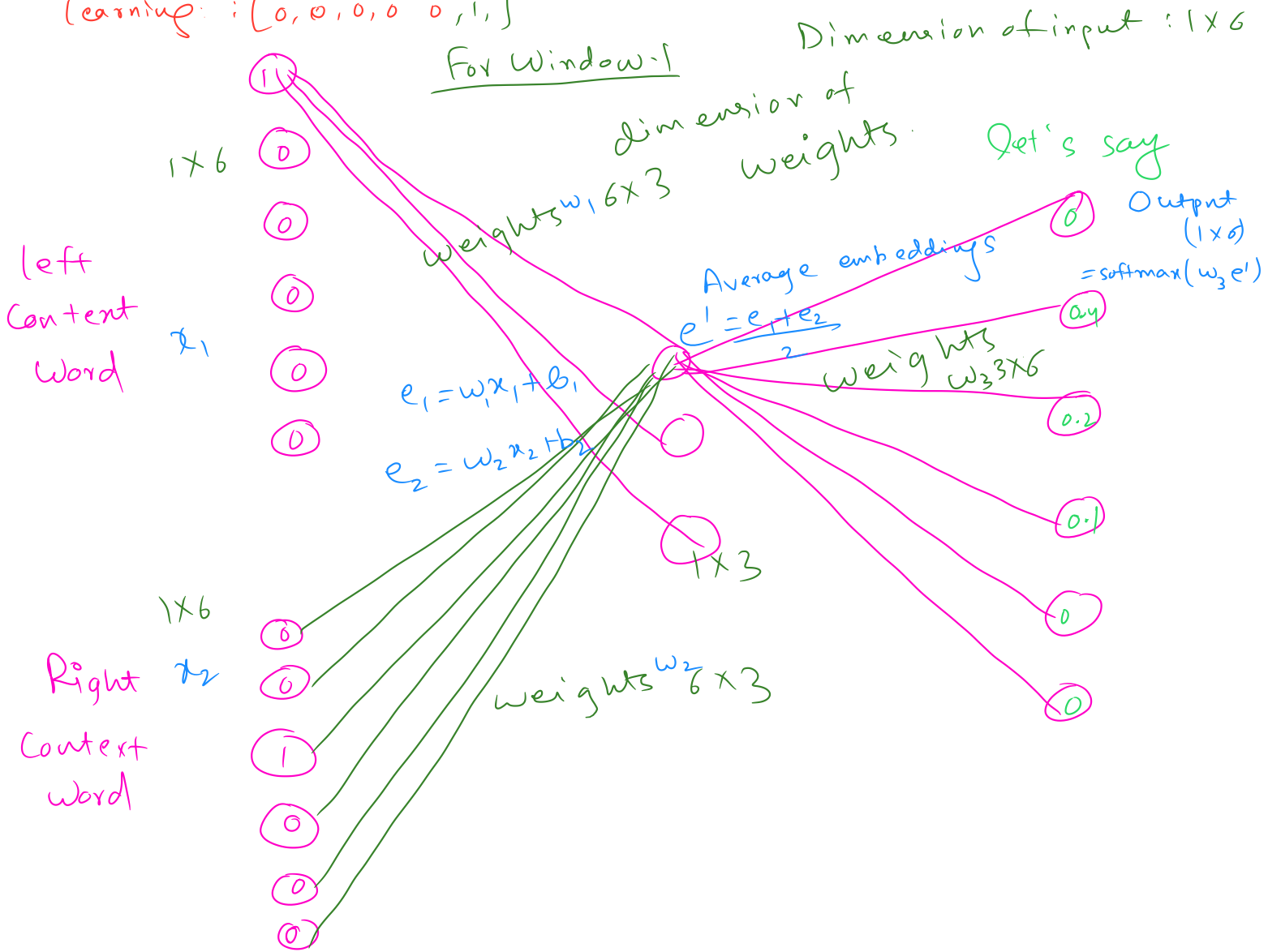
We created a "fill in the blanks" problem "Choose — Intelligence"

So, X (Input) | Y (target)
 Choose, Intelligence | Architecting

It is Supervised ML
 By taking content words as inputs, we are trying to predict the target word

Now we take OHE for our vocabulary

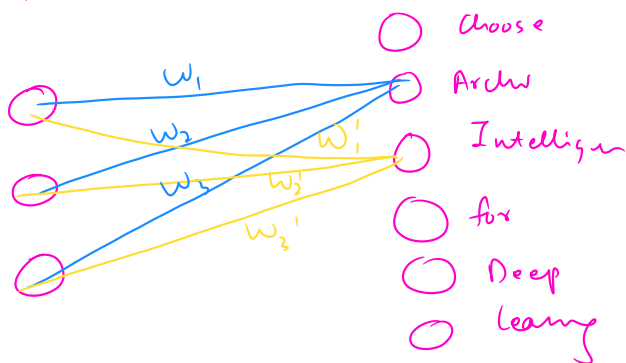
Choose: $[1, 0, 0, 0, 0, 0]$
 Architect: $[0, 1, 0, 0, 0, 0]$
 Intelligence: $[0, 0, 1, 0, 0, 0]$
 for: $[0, 0, 0, 1, 0, 0]$
 Deep: $[0, 0, 0, 0, 1, 0]$
 learning: $[0, 0, 0, 0, 0, 1]$



Expected output $\rightarrow [0, 1, 0, 0, 0, 0]$
 Output we got $\rightarrow [0, 0.4, 0.2, 0.1, 0, 0]$

So, we calculate loss and do back propagation

So, vector representation for words:



After backpropagation is complete
 Architecture: $[w_1, w_2, w_3]$
 We wanted 3 dimensional representation initially
 Intelligence: $[w_1', w_2', w_3']$

Skip gram

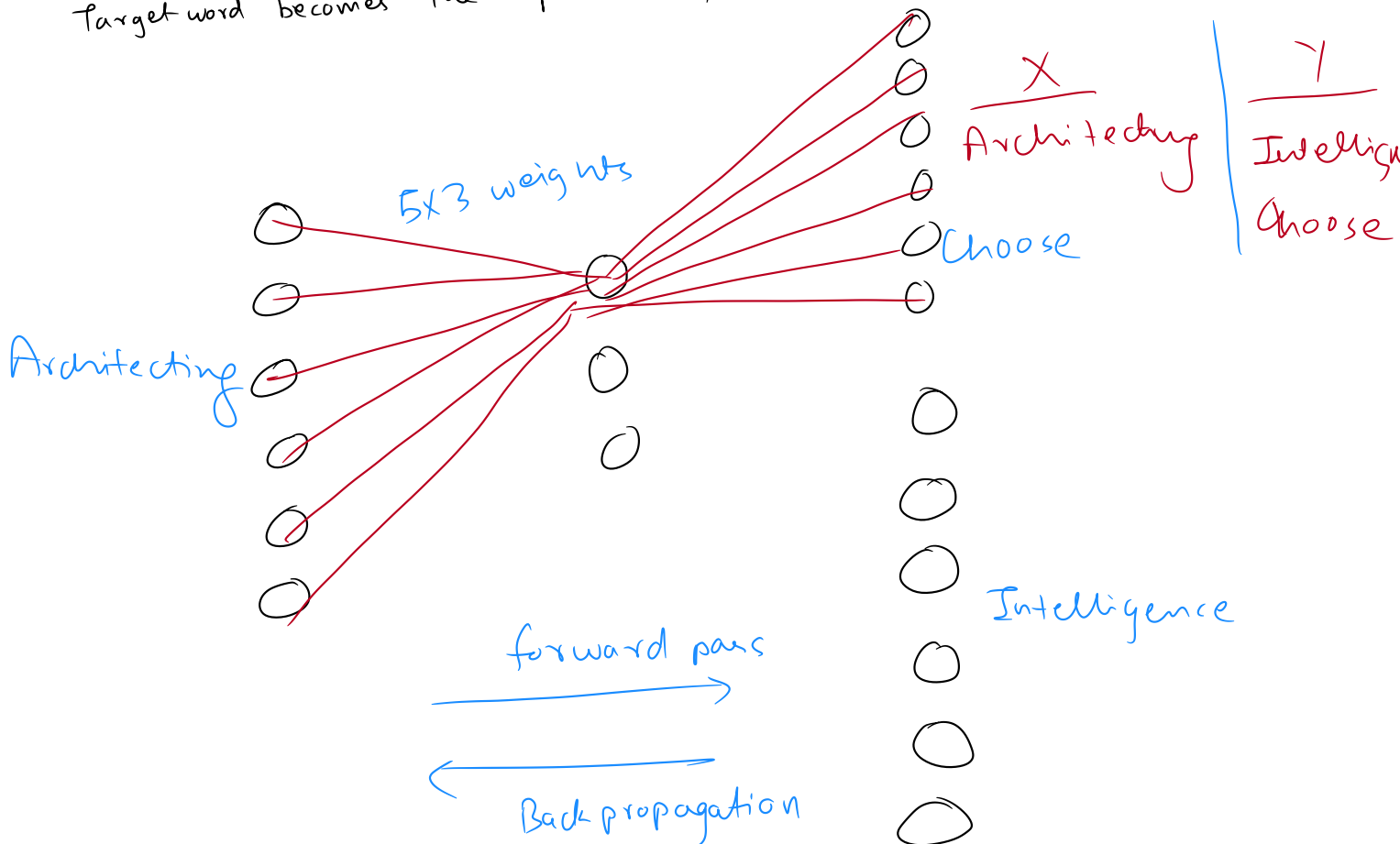
"Choose Architecting Intelligence" for deep learning

Window-size = 3

Reverse our problem this time

"Architecting" window-1
 predicting

Target word becomes the input here, context words become the output



For Small data : use (Bow) } Research proven.
Large data : Skip gram

How can we improve Word2Vec?

- Increase the training
- Increase the dimensions of final vector embeddings (The Hidden layer)
- Increase window size.

New topic Text Classification

Classification: Supervised Machine Learning task

- ↳ On textual data : Email Spam Classifier
Movie Review Sentiment Analysis.

Types of Classification:

Binary Multi Class Multilabel
Eg Entertainment, business.

Single news can be

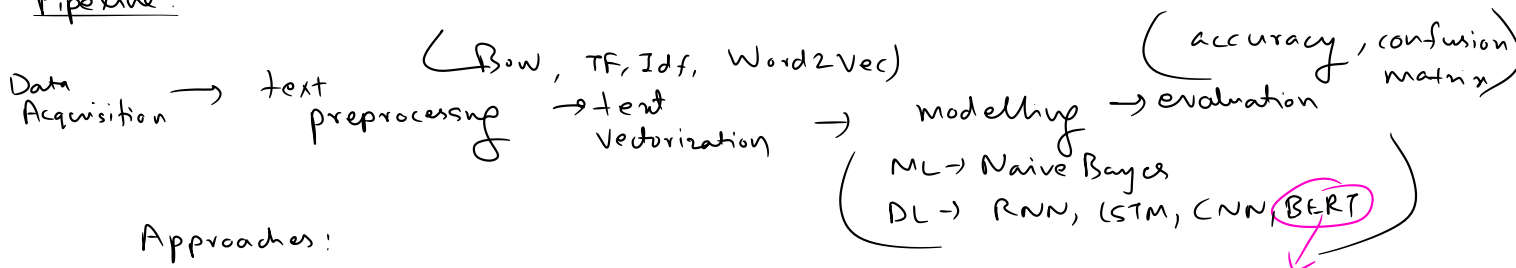
part of multiple Sources.

let's say SRK

→ Bollywood

→ Entertainment.

Pipeline:



Approaches:

Heuristic

APIs (Google, AWS)

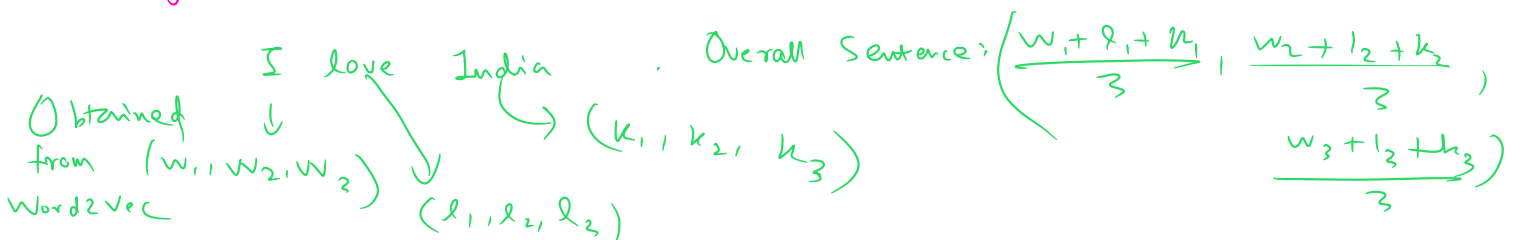
ML : Bow, TfIdf, n-gram

DL : word2vec

pretrained

Word2Vec gives vector embeddings for each word

Avg Word2Vec : for sentence



General Guidelines:

- 1) ensemble techniques instead of Naive Bayes
- 2) Heuristic features should be combined with ML
- 3) Start ML techniques, then go for DL
- 4) Avoid Imbalanced dataset.

(POS)

Parts of Speech Tagging

In simple words, we can say POS tagging is a task of labelling each word in a sentence with its appropriate part of speech.

POS tagging is a preprocessing step.

Used in Named Entity Recognition.

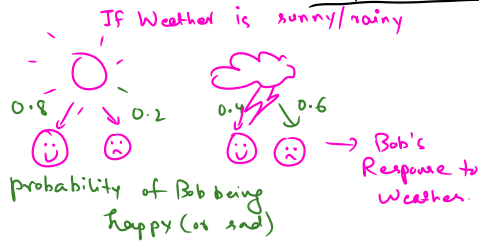
Question Answering System.

Word sense disambiguation

Chatbots.

Check out my Jupyter notebook!

HIDDEN MARKOV MODEL



IF Bob says that he was

Mon Tue Wed Thurs Fri Sat

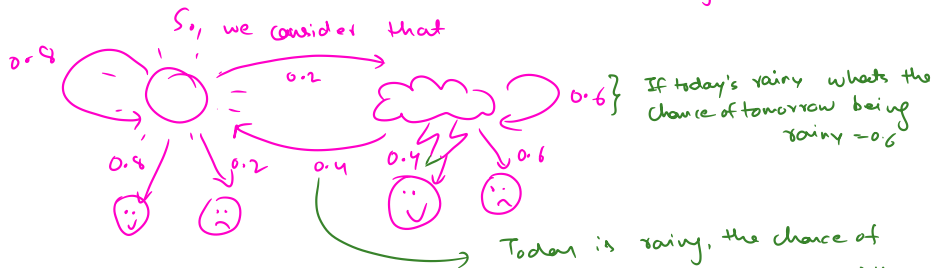
😊 😞 😞 😞 😞 😞

Based on Bob's review we have to guess the weather. So, it can be

S R S R S R

S: Sunny
R: Rainy

But the weather can't change so much.

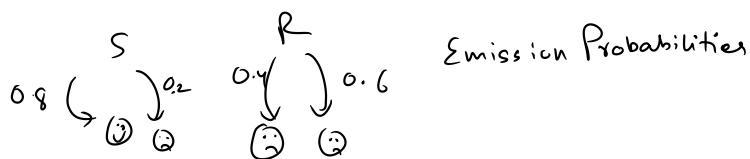
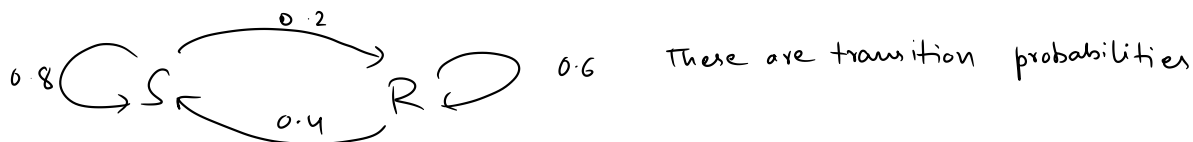


We can only see Bob's reaction, we can't see the whether.

[Bob talks to us over phone and tells us his mood. Bob leaves in France, we live in India]

S₀, Bob's Reactions: Observations.
Weather: Hidden States.

And its called the "Hidden Markov Model"



Q1) How we found these probabilities.
Let's say the data of weather:

S → S → S → S → R → R → R → S → S → S → S → R → R → S → S

How many times S goes to R on the next day?

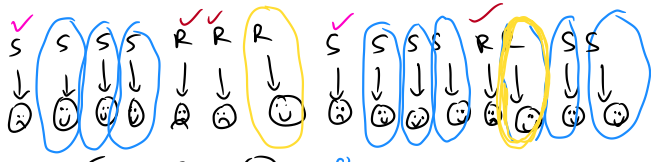
$S \rightarrow R = 2$
 Similarly, $S \rightarrow S = 8$
 $R \rightarrow R = 3$
 $R \rightarrow S = 2$

So, from S

$R = \frac{2}{2+8} = 0.2$
 $S = \frac{8}{8+2} = 0.8$

$R = \frac{3}{2+3} = 0.4$
 $S = \frac{2}{2+3} = 0.6$

Weather to Mood Data:



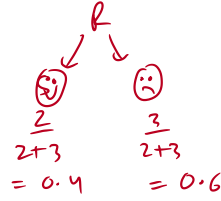
$$S_0, P\left(\frac{\text{Happy}}{S}\right) = \frac{8}{8+2} = 0.8$$

$$S \rightarrow \text{Happy} \rightarrow 8$$

$$R \rightarrow \text{Happy} \rightarrow 2$$

$$S \rightarrow \text{Sad} \rightarrow 2$$

$$R \rightarrow \text{Sad} \rightarrow 3$$



Q2) What is the probability that Random day is Sunny Rainy

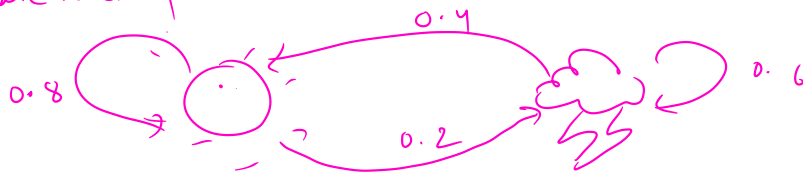
No. of S days = 10

No. of R days = 5

(If Bob gave us no inputs).

$$P(S) = 10/15 = 2/3, \quad P(R) = 5/15 = 1/3$$

Alternate Method



$$S = 0.8S + 0.4R$$

$$S_0, 0.2S = 0.4R$$

$$\boxed{S = 2R} \quad \text{--- (1)}$$

$$S + R = 1$$

$$S = 1 - R$$

$$S_0,$$

$$1 - R = 2R$$

$$R = 1/3, \quad S = 2/3$$

Q3) If Bob says "Happy ☺" ^{he is}

So, Probability of Sunny ↑↑ from 0.67

$$S_0, P\left(\frac{\text{Sunny}}{\text{Happy}}\right) = P\left(\frac{\text{Happy}}{\text{Sunny}}\right) \cdot \frac{P(\text{sunny})}{P(\text{Happy})}$$

$$= (0.8) \times \left(\frac{10/15}{10/15}\right) = 0.8$$

(Bayes Theorem)

Q4) If for three days, Bob is "☺ ☹ ☺", what was the

So, we have to guess the weather on all three weather ?? days.

Let's start with 2 days;

Wednesday

Thurs.

☺

☹

Weather

S R

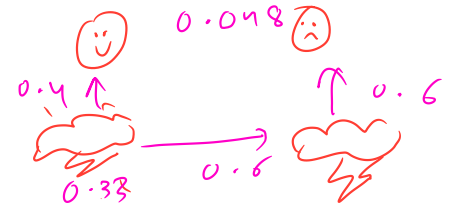
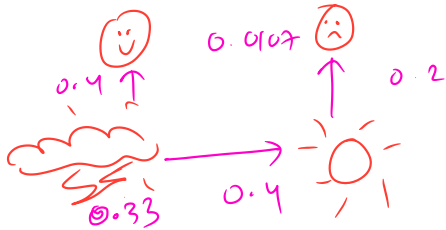
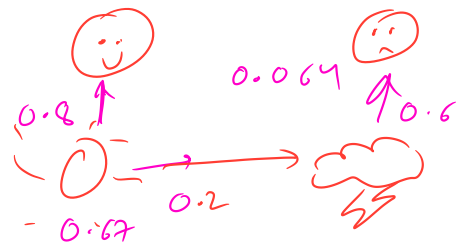
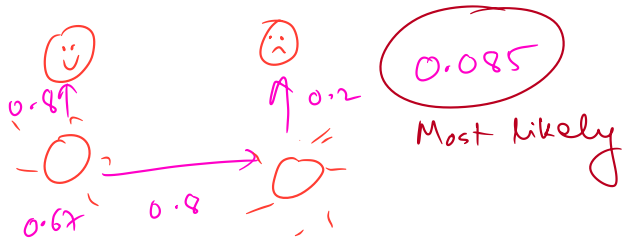
S R

So, 4 combinations

for
Wed
R
R
S

for
Thurs
S
R
S

Scenarios



So, Wed : Sunny
Thurs : Sunny

For three days

Wed Thurs

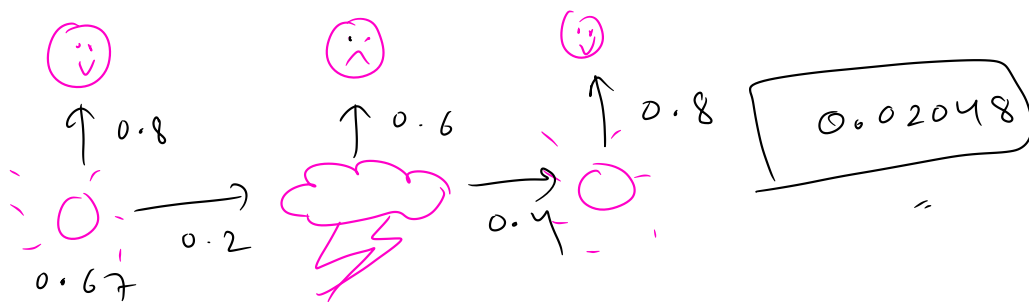
8 combinations

S	S
S	R
S	S
R	R
R	S
R	S
R	R
S	R

Fri

S
S
R
R
S
R
S
R

for Combination



Check all 8 combinations $\rightarrow$ Best combination (SSS)

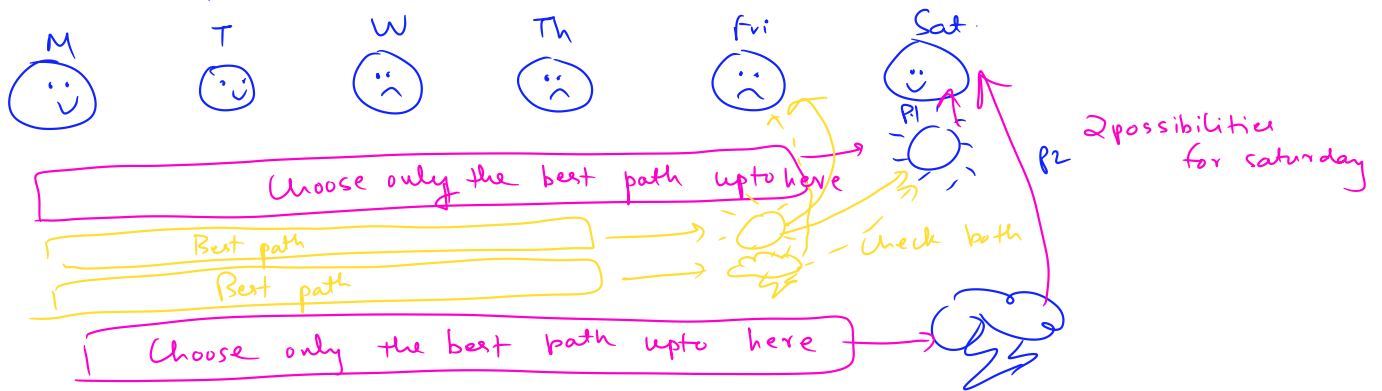
For 4 days, 16 (2^4 combinations)

The number of combinations grows exponentially (2^n).

So, to optimize, we have

VITERBI ALGORITHM

We need to optimize our solution



Check P_1 & P_2 and find out the better probability

Continuing POS tagging.

POS on the sentence

"Will will google Caribbean"

Data

S Jack loves Caribbean E
S Can Jack google Caribbean E
S Will Elizabeth google Caribbean E
S Elizabeth loves Will E
S Will lover google E

S: Start

N: Noun

V: Verb

M: Model

E: End

$$P\left(\frac{\text{Jack}}{\text{Noun}}\right) = \frac{2}{2+3+1+2+2} = \frac{2}{10} = 0.2$$

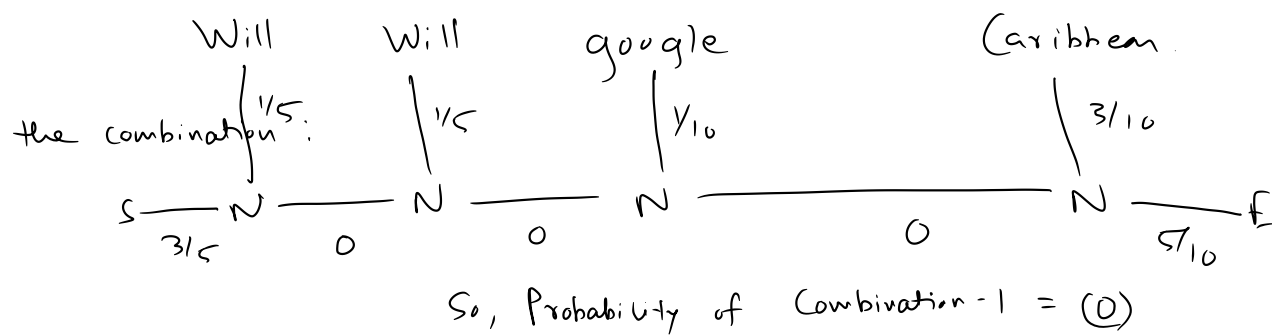
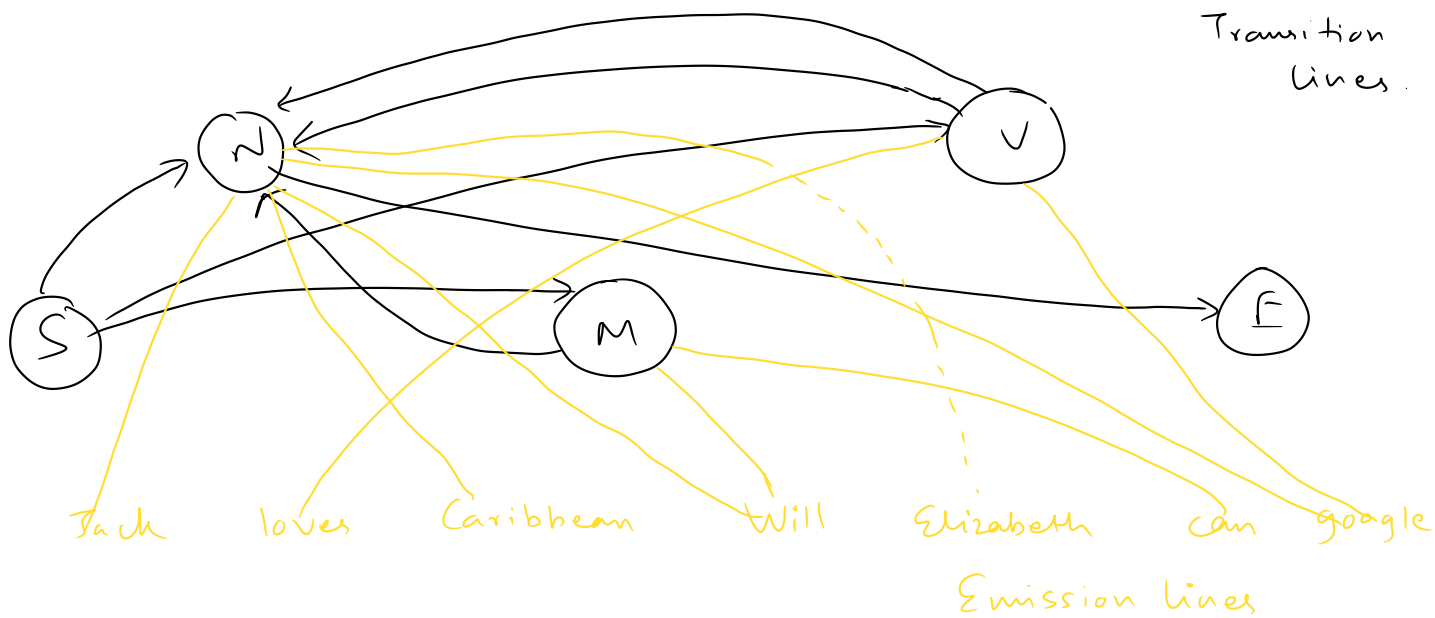
Jack
lover
Caribbean
google
will
Elizabeth
Can

(2)	1/10	0/2	0
	0/10	0/2	3/5
	3/10	0/2	0
	1/10	0/2	2/5
	2/10	1/2	0
	2/10	0/2	0
	0/10	1/2	0

(Emission Probabilities are obtained)

For Transition Probabilities

Transitions	N	M	V	E
S	3/5	2/5	0	0
N	0/10	0/10	5	5
M	2/2	0	0	0
V	5/5	0	0	0



Check all:



Probability of best combination

$$= 0.4 * 0.5 * 0.2 * 0.4 * 0.5 * 0.3 * 0.5$$

$$= \boxed{0.012}$$

So, total possible combinations = $(3^6) = (81)$

But no. of POS $\gg 3$

So, its optimized by Viterbi

